TECHNICAL REFERENCE

Storage Rack

Data Tables, Modules, Services & API Surface

Inventory & storage-location management application

Client-side single-page app set (vanilla HTML / CSS / JavaScript)

Generated 2026-10-15

1. Architecture Overview

Storage Rack is a fully client-side application — there is no backend server, database, or network API. All data is persisted in the browser's `localStorage` as JSON, keyed by a set of versioned keys (e.g. `srStorageV4`). Pages communicate only through that shared storage and a small set of global callback hooks (`window.onRackDataChanged`, `window.onItemDataChanged`) that each page's render function subscribes to.

> Because there is no server, the sections below use "Tables" to mean the **localStorage data collections** that stand in for database tables, and "APIs" to mean the **internal JavaScript function surface** that each page module calls into `common.js` — there are no REST/HTTP endpoints in this project.

Two views, one shared data/logic layer

The app now ships two separate front ends — a **Web view** (sidebar navigation, desktop-first) and a **Mobile view** (bottom tab bar, touch-first, includes the camera QR scanner) — living in their own top-level folders, `/web` and `/mobile`. A single root `index.html` acts as a view chooser: before any login happens, it asks "Mobile View or Web View?" and sends the visitor to `/mobile/index.html` or `/web/index.html` accordingly.

> The two folders share everything that isn't presentation: `js/common.js` (the entire data layer and business logic), every page-specific script (`items.js`, `storage.js`, `storage-detail.js`, etc.), `js/toast.js`, `js/theme.js`, all vendor libraries, and `css/style.css` (every component style — cards, tables, modals, badges, forms). Only the outer shell differs: web pages wrap their content in `.app-shell` / `.sidebar`; mobile pages wrap the exact same content (same element IDs, so the exact same page scripts work unmodified) in `.m-shell` / a top app bar / a bottom tab bar, styled by the mobile-only `css/mobile.css`. A bug fix or business-rule change in `common.js` or any page script therefore applies to both views automatically — nothing about the data model or CRUD behavior is duplicated.

FOLDER	CONTENTS
`/` (root)	`index.html` — the Mobile/Web chooser shown before login. Also still holds the shared `css/`, `js/` assets used by both views.
`/web`	Desktop/laptop-oriented pages: Login, Overview, Items, Storages, Storage Detail, Item Detail, IN/OUT Log, Locations, Users, Roles. Sidebar navigation. No QR scanner — the team does not need camera access on desktop, so `scan.html` was intentionally left out of this view.
`/mobile`	The same functional pages, re-shelled for touch: a floating bottom tab bar with the app's 4 most-used destinations (Home, Items, Scan, Storages — Scan centered) plus a "More" button. Log / Locations / Users / Roles, preferences, and account actions live behind that More button as a shared bottom-sheet menu embedded in every page (not a separate page — see "More" menu below). Adds `scan.html`, the camera-based QR scanner.

> Both views write to and read from the exact same `localStorage` keys and the same `sessionStorage` login flag — logging in on one view and then switching to the other does not require logging in again, and both views always see the same data, since there is still only one browser's storage behind either of them.

Switching is available in both directions, from both the login screen and inside the app, using two small shared CSS components (`.view-chip`, `.view-switch`) defined once in `css/style.css` rather than duplicated per view:

WHERE	WEB	MOBILE
Login screen	"Web view | Change" chip under the brand, linking to the root chooser (`../index.html`)	"Mobile view | Change" chip, same pattern
Inside the app	"📱 Switch to Mobile View" link at the bottom of the sidebar footer, on every page	"💻 Switch to Web View" link inside the More menu's Account section, on every page

> Every switch link routes through the root chooser rather than jumping straight to a specific page in the other view — "Change" means "let me pick again," and since both views share the same login session, picking either option on the chooser lands the visitor already signed in on that view's dashboard, with no extra login step.

Mobile shell, revision 2 — structural patterns, Storage Rack's own colors

The mobile shell was refined against a reference restaurant-management app the team supplied for structural guidance — its actual `mobile/*.html` pages were read in detail, not just its stylesheet. Only layout/structure patterns were adopted — every color, the background, and the brand identity stayed Storage Rack's own (the reference's warm orange/cream theme was explicitly **not** carried over). The changes, all in `css/mobile.css` plus a new `js/more-menu.js`, scoped so `/web` is completely unaffected:

- **Floating pill bottom tab bar, 4 tabs + More** — the bottom nav is a rounded, shadowed pill inset from the screen edges, holding only the 4 most-used destinations (Home, Items, Scan, Storages — Scan centered) plus a More button, rather than trying to fit every page in. The active tab shows as a solid `var(--primary)` pill around its icon.
- **"More" is a shared bottom-sheet menu, not a page** — matching the reference exactly: every mobile page embeds the identical `#moreMenuOverlay` modal (IN/OUT Log, Locations, Users, Roles & Permissions, the theme toggle, the ID quick-search that used to live in the topbar, "Switch to Web View", and Logout). It opens instantly from either the bottom tab bar's More button or the topbar's user-chip — both wired up by the new `js/more-menu.js` — instead of a full page navigation. The `mobile/more.html` page from the first pass was deleted.
- **Topbar simplified** — now just the brand mark/name on the left and a single tappable user-chip on the right that opens the More menu. No page title in the bar (the page's own `.view-head h2` already carries that) and no inline search row — both were removed to match the reference's clean, uncluttered topbar.
- **Bottom-sheet modals everywhere** — every modal on every mobile page (`.m-view .modal-overlay` , including the new More menu) slides up from the bottom edge with rounded top corners, instead of the centered desktop-style dialog web still uses. One CSS rule covers all of them, since they already share the same `.modal-overlay` / `.modal` markup.
- **Toasts repositioned** — lifted above the floating tab bar and widened to the full screen width on mobile, so they no longer sit underneath/behind the nav.
- **Bolder page headings** — `.view-head h2` (page titles like "Overview", "All Items") is larger on mobile, matching the more app-native typographic scale from the reference.
- **Login screen** — top-aligned, app-native layout: brand header, a "Mobile view | Change" chip (links back to the root chooser), a bold "Welcome back" heading, icon-prefixed inputs with a working show/hide password toggle, a pill Login button, a dashed demo-credentials box, and a footer line. See the Login module card below for the full description.
- **View chooser cards** — the root chooser's Mobile/Web options were upgraded from plain rows to icon-badge cards with a trailing arrow, matching the reference's option-card pattern.

Full responsiveness — phones, tablets, laptops, desktops, for both views

Both `/web` and `/mobile` are audited and fixed across the real device range: 320px (the smallest phones) through 1920px (desktop), not just the two sizes each view was originally designed around.

- **Fixed a real overflow bug** — `.topbar-search` and its input didn't have `min-width: 0` , so at the narrowest phone widths (320px) the browser's default intrinsic minimum width for a text input held the search bar wider than the screen, overflowing it. The same `min-width: 0` fix already used elsewhere in this codebase (e.g. `.main-area`) applies here too.
- **The mobile view is now usable on laptops/desktops too, as a phone-frame mockup** — opening `/mobile` directly on a wide screen (or switching to it from `/web`) used to stretch every page edge-to-edge, which looked broken: stat cards spread absurdly wide, the floating tab bar looked disconnected from the content. At `min-width: 560px` , both `.m-shell` and the login screen's `.m-login-shell` are now constrained to a centered, phone-width (480px) card framed like a device mockup — a thick rounded bezel (`10px` border, `44px` corner radius) with a decorative notch pill at the top, floating with margin on every side instead of touching the viewport edges — matching the structural border/margin pattern of the reference app's device mockup, built entirely from Storage Rack's own border/shadow tokens rather than the reference's colors. The topbar's own top corners get a matching radius so its square background doesn't poke past the frame's curve (chosen over `overflow: hidden` on `.m-shell` , which would have turned it into a new sticky-positioning container and broken the topbar's `position: sticky`). The frame's exact dimensions (390px wide, height `min(844px, 100vh - 48px)` , 12px bezel, 54px corner radius, a 110×30 notch) are matched to the reference app's own device-frame rule (`D:/restaurant_app` , `assets/css/style.css` , its `@media (min-width: 521px)` `.m-phone` block) rather than an

approximation — only the bezel color differs, using the app's own `var(--border-strong)` instead of the reference's literal brown/black. `body.m-view` centers the frame both horizontally and vertically on the page (`display: flex; align-items: center; justify-content: center;`) now that it's capped at a fixed phone size instead of always filling the viewport. Inside the frame, `.m-shell` / `.m-login-shell` scroll internally (`overflow-y: auto` , scrollbar hidden via `scrollbar-width` / `::-webkit-scrollbar` so it doesn't eat into the frame's width) instead of growing taller than the viewport and letting the page itself scroll — matching how a real device mockup behaves (a fixed-size screen, scrolling content). The bottom tab bar and toast container stay `position: fixed` to the viewport (not `absolute` relative to the frame) specifically so they don't scroll away with the frame's internal content — a descendant of a scrolling element keeps scrolling with it even as `position: absolute` , since its containing block only fixes its coordinate origin, not an exemption from the ancestor's scroll; only `position: fixed` , anchored to the viewport, escapes that. Because the frame is capped at 844px tall and centered, its distance from the viewport edges isn't constant once the viewport is taller than that — so CSS custom properties, `--frame-gap` (vertical) and `--frame-left` (horizontal), compute the leftover space around the frame at the current viewport size, and every position: fixed overlay's offsets are expressed in terms of them instead of fixed pixel values, so they stay correctly docked to the frame's actual edges at any window size. This includes modals: the More menu, add/edit forms and the lightbox are appended outside `.m-shell` and are `position: fixed` , so without this they'd cover the full browser window instead of staying inside the phone mockup — `.m-view .modal-overlay` 's `inset` is constrained to the frame's own outer bounds (`top` / `bottom: var(--frame-gap)` , `left` / `right: var(--frame-left)` , with a matching 42px corner radius, matching the inner screen curve rather than the bezel's outer 54px) instead of the default `inset: 0` . `--frame-gap` / `--frame-left` themselves locate the frame's *outer* edge, so the overlay's insets add the 12px bezel width on top of them (`calc(var(--frame-gap) + 12px)` , and `--frame-left` is computed directly against the 366px inner width) — using the outer values directly, as a first pass did, let the overlay cover the bezel itself instead of stopping at the inner screen edge. Its own max-height is capped at 88% of the inner screen height (matching the reference app's own `.modal-box { max-height: 88%; }`), not the full inner height — otherwise a modal whose content needs scrolling (the More menu, on most window sizes) grows to fill nearly the entire frame, leaving no gap above it showing the page underneath, which reads as a full-screen page takeover rather than a bottom-sheet overlay. A modal that reaches all the way to the frame's bottom edge also keeps its normal bottom-sheet corners (square, since on an actual phone it's flush with the screen's own square bottom edge) unless rounded to match — `.m-view .modal` 's bottom-left corner gets the same 42px curve as the topbar and tab bar so it sits inside the bezel instead of poking past it. The bottom-right corner is deliberately left square: a vertical scrollbar is always a straight bar flush against that edge, so rounding it just traded "square corner past the bezel" for "scrollbar cutting through the curve." Left square, the only cost is a small sliver of the frame's own bezel visible behind that one corner — far less noticeable than either alternative. Unlike `.m-shell` , a modal's own scrollbar is *not* hidden outright: on a shorter browser window the More menu's full content (search, two nav groups, theme toggle, account actions) can exceed the frame's available height, and with no scrollbar at all a viewer has no way to tell the menu is scrollable rather than simply cut off after "Dark Mode." It's shown instead as a slim, themed bar (6px, `var(--border-strong)` thumb, transparent track) rather than the bulky native one. That let the bottom tab bar switch from a floating inset pill (the right look on an actual phone, which has no bezel to dock against) to a bar docked flush edge-to-edge at the very bottom of the frame — full inner width, rounded only on its bottom corners to continue the bezel's curve, pinned at a fixed `bottom` offset that lines up exactly with the frame's inner-bottom edge. The toast container (appended to `<body>` , not inside `.m-shell` , and also `position: fixed`) got the same centering treatment, sized to the frame's inner content width (480px frame − 10px bezel on each side = 460px) rather than its outer width.

- **Container queries, not just viewport media queries, for content reflow** — constraining the mobile card's width exposed a deeper issue: rules like `.rack-row` stacking, the `.log-table` → stacked-card collapse, and `.stats-row` 's column count were all written as `@media (max-width: …)` , which reacts to the **browser viewport**. On a wide desktop screen showing the mobile view's narrow 480px card, those rules never fired — the viewport itself was wide — so a 7-column table tried to render inside a 480px box and visibly overlapped. Fixed at the root: `.main-area` (web) and `.m-shell` (mobile) are now both named CSS containers (`container: app / inline-size;` , exactly the technique the reference app uses for the same reason), and the affected rules were converted to `@container app (max-width: …)` — they now respond to the actual rendered width of the content area, correctly reflowing regardless of whether that width comes from a narrow phone, a narrow desktop window, or a 480px card sitting inside a 1920px viewport. Purely window-chrome rules (the sidebar's off-canvas collapse, the web topbar's own padding) correctly stayed as ordinary viewport media queries, since those are about the browser window itself, not content width.

2. Data Tables (localStorage collections)

Six JSON collections make up the entire data model. Each is loaded once per page load, held in a module-level variable, and re-saved on every mutation.

2.1 Storage Units — `racks` (key: srStorageV4)

FIELD	TYPE	DESCRIPTION
id	string	Unique ID, e.g. `ST-1008`
name	string	Display name, e.g. "Chem Storage"
storeRoomId	string	FK → Locations.id
type	string	Storage type (rack, cupboard, bureau, wardrobe, shelf, cabinet, box, custom)
tags	string[]	Free-form category tags
images	string[]	Base64 data-URI pictures
nodes	array	Legacy sub-structure tree (deprecated — storages are flat; items attach directly)
createdAt	number	Epoch ms

2.2 Items — `globalItems` (key: srGlobalItemsV1)

FIELD	TYPE	DESCRIPTION
id	string	Unique ID, e.g. `IT-1013`
name	string	Item name
tags	string[]	Category tags
quantity	number	Units on hand
remarks	string	Free-text notes
storageId	string | null	FK → Storage Units.id (null = unassigned)
images	string[]	Base64 data-URI pictures
status	string	`in_store` | `in_use` | `removed` (archived)
createdAt	number	Epoch ms

2.3 Locations — `rooms` (key: srRoomsV1)

FIELD	TYPE	DESCRIPTION
id	string	Unique ID, e.g. `SR-1002`
name	string	Location name
location	string	Physical area / warehouse
createdAt	number	Epoch ms

2.4 Users — `users` (key: srUsersV1)

FIELD	TYPE	DESCRIPTION
id	string	Unique ID, e.g. `US-1002`
name	string	Full name
username	string	Login handle
email	string	Contact email — now required; it's the only channel the "Send Invite" / password-reset flow has to address the user
phone	string	Contact phone number — optional
roleId	string	FK → Roles.id
status	string	active | inactive — inactive users are rejected at login even with a correct password
password	string | undefined	Plaintext demo password; not settable from the Users page UI itself — that only sends the invite/reset. Once a value exists here (however it got set), the user can sign in with `username` + this value
createdAt	number	Epoch ms

2.5 Roles & Permissions — `roles` (key: srRolesV1)

FIELD	TYPE	DESCRIPTION
id	string	Unique ID, e.g. `RL-1002`
name	string	Role name
description	string	Free-text summary
permissions	object	Per-module CRUD matrix — keys: storage, items, rooms, users, roles; each holds {create, read, update, delete: boolean}
createdAt	number	Epoch ms

2.6 IN/OUT Log — `itemLog` (key: srItemLogV1)

FIELD	TYPE	DESCRIPTION
id	string	Unique log entry ID
timestamp	number	Epoch ms
type	string	in | out | in_store | in_use | removed
itemName	string	Item name at time of entry
quantity	number	Units moved
rackId / rackName	string	Storage unit involved (denormalized)
path	string | null	Legacy sub-structure path (from the deprecated nested-compartment tree); every call site in the current codebase passes `null`, so this field is effectively dead today but still present on every row
user	string	Acting user's *username* (from `sessionStorage`) — not a foreign key to Users.id

3. Modules (Pages)

Each HTML page pairs with one page-specific script, on top of the shared `common.js`, `toast.js` and `theme.js` loaded everywhere. Every module below (other than the chooser, Scan QR, and More) exists as two files — `/web/X.html` and `/mobile/X.html` — that both load the **exact same, unmodified** page script; only the surrounding shell markup differs between the two.

View Chooser

`/index.html (root)`

The very first screen, shown before any login. Two buttons — "Mobile View" and "Web View" — send the visitor to `/mobile/index.html` or `/web/index.html`. Carries no app logic of its own and loads none of the shared data layer.

Login

`web/index.html · mobile/index.html · js/login.js`

Demo credential sign-in, "forgot password" and "reset demo data" flows — identical logic and the same colors/theme on both views (shared `login.js`, shared CSS variables). The two views differ only in layout: web keeps the centered desktop login card; mobile uses a top-aligned, app-native screen — brand header, a bold "Welcome back" heading, icon-prefixed username/password fields with a show/hide password toggle, a pill-shaped Login button, a dashed-border demo-credentials box, and a footer line. The password toggle (`togglePasswordBtn`) is wired up in `login.js` behind a guard, so it's a no-op on the web login where that button doesn't exist. Both logins now show a "{view} view | Change" chip (`.view-chip`, shared CSS) just under the brand — mobile's says "Mobile view", web's says "Web view" — linking back to the root chooser so a visitor who picked wrong can switch before even signing in. Beyond the single hardcoded `admin` / `admin123` credential, `login.js` now also checks the submitted username/password against `srUsersV1` directly (reading the key from `localStorage` rather than loading `common.js`, keeping this page self-contained): any user with a `password` on their record and `status: "active"` can sign in with their own credentials, landing on the same post-login page as the demo admin account. The Users page's invite/reset action (see the Users module card) only sends the request — it doesn't set this field itself.

Overview (Dashboard)

`web/dashboard.html · mobile/dashboard.html · js/overview.js`

Summary stat cards, empty-storage alerts, recently added storages. Empty-state padding was tightened app-wide (60px → 32px) so panels with no content (e.g. "Empty Storage Units" when there are none) no longer reserve an oversized, unattractive blank box. Each row's `.rack-id-badge` (shared with the Storages card grid) now has a `min-width: 58px` and centered text instead of hugging its own text — it previously sized to each row's own ID length, so a short custom ID (e.g. a storage entered as just "513" instead of the usual `ST-1008` pattern) shrank the badge and shifted that one row's name out of alignment with every other row in the same list. A new `.rack-row-header` row ("ID" / "Name" / "Location" / "Type" / "Items", styled like the Items/IN-OUT Log tables' column headers) sits above both the "Empty Storage Units" and "Recently Added" lists on web. `.rack-row` itself was restructured from a 3-column grid (ID + name + location grouped as one flex block, then type, then items) to 5 independent grid columns (`74px 1fr 120px 150px 150px`) so ID, Name and Location each line up as their own column too, not just Type and Items — `buildRackRow()` in `common.js` now emits five direct grid children instead of wrapping the first three in a `.rack-row-main` flex block. Splitting out five fixed/flexible columns raised the row's minimum width, so the fixed-column widths were tightened (sum ≈494px instead of the old ≈410px) to keep that minimum comfortably under the existing 700px collapse breakpoint rather than raising the breakpoint itself. It's web-only: mobile's rows always collapse to a single stacked column (`@container app (max-width: 700px)`, which mobile's narrow shell is permanently inside), where column headers wouldn't correspond to anything, so the markup isn't duplicated there. The header hides itself along with the row list for the "every storage unit has at least one item" empty state.

Items

`web/items.html` · `mobile/items.html` · `js/items.js`

Full items table: filter by Type and Status, sortable columns (including a dedicated Item ID column), search across item name, item ID, storage name, storage ID, tags and location, add/edit modal, archive & restore, CSV/XLSX import, CSV/PDF export. Tags render as colored chip widgets (matching Status's badge styling) rather than plain text. The item's picture has its own dedicated first column instead of being embedded inline with the name; items with no picture show a colored letter-avatar (the item name's first letter) in that column instead. On mobile, the existing responsive table-to-card collapse (already built into `style.css`) turns each row into a stacked label/value card automatically — no mobile-specific table code was needed.

Storages

`web/storage.html` · `mobile/storage.html` · `js/storage.js`

Storage unit card grid: filter/sort/search, pagination, add/edit/delete. Every card is a fixed, uniform height (260px) regardless of content — both the item-chip preview (`buildItemsPreviewHtml`, capped at 2 + "+N more") and the tag list (`rackCardTagsHtml`, capped at 2 + "+N") are kept tight so sparse cards (1 tag, 1 item) don't leave excess dead space while cards with lots of items/tags still truncate safely inside the same fixed box. The "N items" footer is pinned to the bottom of every card via a corrected flex chain (`.rack-card-body` now properly stretches so `.rack-meta`'s `margin-top: auto` — previously a no-op — actually pushes it down).

Sort: Name (A-Z / Z-A) | Sort: Newest First | Sort: Oldest First | Sort: Location (A-Z) | Sort: Type (A-Z) | Sort: Items (most / fewest)

Scan QR (mobile only — intentionally absent from web)

`mobile/scan.html` · `js/scan.js`

Camera-based QR scanner, built for opening a storage unit's full detail page by scanning the same QR code the Storage Detail page prints/displays. Requests the device camera (rear-facing preferred via `facingMode: "environment"`), continuously grabs video frames to an offscreen canvas, and decodes them with `jsQR`. The app's QR codes encode an absolute URL (e.g. `.../mobile/storage-detail.html?id=ST-1008`); a decoded result is parsed for its `id` query parameter (regardless of which view's URL path it carries), looked up via `findRack`, and on a match the page redirects straight into that view's Storage Detail for that unit. A manual ID-entry field is always visible as a fallback. Camera access requires a secure context (HTTPS, or localhost for local testing). There is no web equivalent — desktops/laptops don't need camera-based lookup, so `scan.html` was deliberately left out of `/web` and out of its bottom-nav-less sidebar entirely.
It's mobile's home screen: logging in lands here directly (`js/login.js`'s post-login redirect is `scan.html` on mobile, still `dashboard.html` on web, detected via whether the page has a password-visibility toggle), and `Scan` sits in the center of the bottom tab bar (Home, Items, **Scan**, Storages, More) rather than off to one side. The page itself is two cards — a "Quick Scan" card (icon badge, heading, description, a pill "Tap to Scan QR" button) and a "Search by ID" card below it — instead of one plain block.
Tapping "Tap to Scan QR" opens a fullscreen scanner overlay (`#scanOverlay`, new `js/scan.js` helpers `openScanOverlay()` / `closeScanOverlay()`) rather than showing the camera preview inline in the card — a "QR CODE SCANNER" title, the live video inside a square frame marked by four corner brackets (not a full border) with an animated scan line, the same heading/description repeated, a status pill ("Align QR code within the frame", turning green with the matched storage's name on a hit), and a Cancel button that stops the camera and closes the overlay. It follows the same precedent as the image lightbox (`.lightbox-overlay`, a separate class rather than reusing `.modal-overlay`) and so is NOT constrained to the phone-frame mockup on wide screens — a true fullscreen camera view is expected to cover the whole browser window, the same way a real device's camera UI would. The corner brackets turn `var(--success)` green on a match, matching the reference's visual language while keeping Storage Rack's own blue `var(--primary)` instead of its colors. The manual ID search's "not found" case now surfaces as a toast rather than inline status text, since that status element lives inside the overlay and the search field doesn't open it.

"More" menu (mobile only — a shared modal, not a page)

`#moreMenuOverlay in every mobile/*.html · js/more-menu.js`

The bottom tab bar only has room for the app's 4 busiest destinations (Home, Items, Scan, Storages — Scan centered), so IN/OUT Log, Locations, Users, and Roles & Permissions are grouped behind a More button instead of each getting their own tab — matching the reference app exactly, including that "More" opens an in-page bottom-sheet modal rather than navigating to a separate page. The identical modal markup is embedded on every single mobile page; `js/more-menu.js` wires it up to open from either the bottom tab bar's More button or the topbar's user-chip, and to close via its own close button or a backdrop tap. It also hosts the ID quick-search (moved here from the topbar), the theme toggle, and "Switch to Web View" / Logout. Its nav links reuse the same `.nav-item[data-route]` markup as the tab bar, so `common.js`'s existing icon-swap and active-state logic (`initSidebar()`) highlights the current page with no extra code — including a small addition that treats the bottom tab bar's own More button as "active" whenever the current page is one of the four menu-only pages.

Storage Detail

`web/storage-detail.html · mobile/storage-detail.html · js/storage-detail.js`

Single storage's page: per-field click-to-edit (name, location, tags, pictures), always-on QR code with print, items panel with inline row editing plus its own search, status filter and sortable column headers (Item, Tags, Status, Qty). Panel's Tags column renders as colored chip widgets, matching Status's badge styling. Items Stored, Type and Tags are each shown as their own separate widget card, matching the Item Detail page's layout. The panel also has a dedicated first Image column — a real thumbnail when the item has a picture, otherwise a colored letter-avatar — matching the Items page.

Item Detail

`web/item-detail.html · mobile/item-detail.html · js/item-detail.js`

Single item's page: per-field click-to-edit (name, storage, tags, quantity, remarks, pictures). Quantity, Status and Tags are each shown as their own separate widget card, now spanning the full row width evenly (no more capped-width cards leaving dead space); activity history with filters; archive/restore.

IN/OUT Log

`web/log.html · mobile/log.html · js/log.js`

Full movement history: filter/sort/search, CSV export. Reached directly from the sidebar on web; reached via the "More" tab on mobile.

Locations

`web/storerooms.html · mobile/storerooms.html · js/storerooms.js`

Manage physical locations storages belong to; add/edit/delete. Reached via "More" on mobile.

Users

`web/users.html` · `mobile/users.html` · `js/users.js`

User directory with role assignment; add/edit/delete. Each row has a colored letter-avatar (first letter of the user's name) as its first column, for visual identity consistent with the Items table. Reached via "More" on mobile. The Add/Edit User modal has a required Full Name, a Role selector with an inline "+ New Role" link to the Roles page, Username, optional Phone, and a now-**required** Email — email is mandatory because it's the only addressable channel the password-reset flow has. A "Send Invite" action (alongside Cancel and Save) first validates and saves the user record (creating it if the form was still in "Add User" mode), then opens a `confirmDialog` ("Send a password reset link to [name]?") — confirming it shows a single "Invitation sent to [email]" toast, and that's the entire admin-side flow. The table's key-icon action sends the same invite directly for an existing user, without opening the Add/Edit form. Neither one shows a password field anywhere in this UI: setting the actual password is modeled as something that happens only on the recipient's end, by following the link in their email — not something the person sending the invite ever sees or sets on their behalf (see the Login module card for how a `password`, however it was set, lets that user sign in). If the user has no email yet, the key-icon action redirects to Edit User instead, with a toast explaining why, since there would be nothing to "send" to. No real email is ever sent — the invite is just a toast, consistent with the rest of the app's demo-only auth.

Roles & Permissions

`web/roles.html` · `mobile/roles.html` · `js/roles.js`

CRUD permission matrix per module; add/edit/delete roles. Reached via "More" on mobile.

Shared infrastructure

FILE	PURPOSE
js/common.js	Data layer, icon system, shared UI builders, modals (confirm, add item, quick-add storage/location, lightbox) — loaded by both /web and /mobile pages unchanged
js/toast.js	Toast notifications + cross-page flash messages
js/theme.js	Light/dark theme toggle, applied before paint
js/vendor/qrcode.js	QR code generation (kazuhikoarase qrcode-generator)
js/vendor/jsQR.js	QR code decoding from camera frames (cozmo/jsQR) — used only by mobile/scan.html
js/more-menu.js	Opens/closes the shared "More" menu modal on mobile pages, from either the bottom tab bar's More button or the topbar's user-chip
js/vendor/xlsx.js	Spreadsheet import/export (SheetJS)
js/vendor/jspdf.js + jspdf.autotable.js	PDF export for item/log tables
css/style.css	Every component style (cards, tables, modals, badges, forms, buttons) — shared by both views; also still holds the old `.app-shell` / `.sidebar` styles used only by /web
css/mobile.css	Mobile-only shell styles: top app bar, bottom tab bar, the "More" page's list/card styles, the redesigned Login screen, and a site-wide pill-shaped `border-radius` override for `.btn-primary` / `.btn-secondary` on mobile. Loaded after style.css so it only overrides shape/layout, never the shared color variables — mobile and web intentionally use the exact same theme/colors

4. Services (Shared Business Logic)

Functions in `js/common.js` that encapsulate data rules and are reused across multiple pages, grouped by concern.

Data persistence (load / save per table)

loadRacks / saveRacks · loadGlobalItems / saveGlobalItems · loadRoles / saveRoles · loadLog / saveLog · loadRooms / saveRooms · loadUsers / saveUsers

Lookups & ID generation

findRack · findGlobalItem · findRoom / roomLabel · nextRoomId / nextUserId / nextRoleId / nextItemCode / nextNodeCode · findRole / roleLabel · findUser · rackDetailUrl / itemDetailUrl · rackTags / rackImages · runIdSearch

Item lifecycle rules

archiveGlobalItem · restoreGlobalItem · purgeGlobalItem · activeGlobalItemsForRack · globalItemsForRack · unitItemCount · logTransaction (in/out) · logStatusChange

Storage deletion & permissions

deleteRack · emptyPermissions · permissionSummary

Shared modals (dynamically built, reused across pages)

confirmDialog · openAddItemModal / closeAddItemModal · openQuickAddStorageModal · openQuickAddLocationModal · openEditModal / closeEditModal · openLightbox / buildLightbox · openMoveItemModal / closeMoveItemModal

Two searchable "item picker" dropdowns exist, both matching on item name, ID, tag, remarks or current storage location, both rendering their results inline below the field (not as an absolutely-positioned overlay, so they can't be clipped by a modal's own scroll container):

- **Add Storage** (`openQuickAddStorageModal`) — a dedicated "Add Existing Items" field: multi-select via removable chips; on submit, every picked item's `storageId` is reassigned to the newly created storage.
- **Add Item** (`openAddItemModal`) — the **Item Name** field itself doubles as the search box (only while creating — suppressed while editing an existing item): typing shows matching existing items below it; clicking, or pressing Enter/arrow-keys to navigate and Enter to confirm, opens a `confirmDialog` ("Move X into Y? It will be taken out of Z.") before reassigning that item's `storageId` to whatever Storage is currently selected in the form — nothing moves until the user confirms. Pressing Enter with the dropdown open always targets the highlighted (or first) match instead of submitting the form, so it can't create an accidental duplicate item. Leaving the dropdown alone and just filling in the form still creates a brand-new item exactly as before.

Reusable UI builders

buildRackCard / buildRackRow · buildItemsPreviewHtml · populateRackSelect / roomOptionsHtml · tagsHtml / categoryBadgeHtml / typeBadgeHtml · renderPagination / paginateArray · itemStatusBadgeHtml · imagesPreviewHtml / rackQrThumbHtml · animateNumber / escapeHtml / svgIcon · createTagInput / createImageInput · getCategoryColor / getItemColor / hashString

5. API Surface (Internal, Client-Side)

> This project has **no network API** — no REST, GraphQL, or server routes exist. The table below documents the de-facto "API" each page module relies on: the shared functions and global callback hooks exposed by `common.js`, which act as the contract between modules.

CONTRACT	DIRECTION	USED BY
`window.onRackDataChanged`	Hook set by the current page's render function; called by common.js after any rack mutation (add/edit/delete, quick-add)	storage.html, storage-detail.html
`window.onItemDataChanged`	Hook set by the current page's render function; called by common.js after any item mutation (add/edit/archive/restore/purge/move)	items.html, storage-detail.html, item-detail.html
`findRack(id) / findGlobalItem(id) / findRoom(id) / findUser(id) / findRole(id)`	Synchronous read API — every page queries in-memory collections through these instead of touching localStorage directly	All pages
`save*(data)` family	Synchronous write API — the only sanctioned way to persist a table back to localStorage	All pages
`open*Modal(...)` family	UI API — pages invoke these instead of building their own modal markup, keeping modal behavior consistent app-wide	items.html, storage.html, storage-detail.html, item-detail.html
`confirmDialog(message, onConfirm, options)`	UI API replacing native `confirm()` everywhere in the app	All pages with destructive actions
`showToast(message, variant)` / `setFlashMessage`	Notification API (toast.js) — immediate toast or a flash message persisted across a redirect	All pages

6. Backend Development Guide

The app is currently 100% client-side (Section 1) — all six collections in Section 2 live only in one browser's `localStorage`, so nothing is shared between users or devices, and the "login" in `js/login.js` is a single hardcoded demo credential (`admin` / `admin123`) with no real session, token, or server-side permission check anywhere. The Roles & Permissions CRUD matrix (Section 2.5) is fully modeled and editable in the UI today, but nothing currently reads or enforces it — it's display-only until a backend exists to check it on every request. This section proposes a Django backend that preserves the existing data shapes (Section 2) and function contracts (Section 4–5) as closely as possible, to minimize required frontend rewrite.

> The backend is split across **two Django apps** rather than one: `accounts` owns everything behind the Users and Roles pages (the `Role` and `User` models, login/invite, and the permission check both apps rely on), and `storage_rack` owns everything else (Locations, Storage Units, Items, and the IN/OUT Log). Neither app's models reference the other's — the only place they touch is a single import, `storage_rack` pulling `HasModulePermission` from `accounts.permissions`, since permission checking is inherently about Roles. Splitting this way keeps "who can log in and what can they do" independently deployable, testable, and migratable from "what's actually in the warehouse."

6.1 Django Models

An **exact** field-for-field mirror of the six tables in Section 2 — every field name, type, and nullability matches the frontend's actual JSON exactly, with no added/dropped/upgraded fields, split across the two apps: `accounts/models.py` holds `Role` and `User`; `storage_rack/models.py` holds `Location`, `StorageUnit`, `Item`, and `ItemLogEntry`. `id` stays the same string the frontend already generates (e.g. `ST-1008`, `IT-1013`) as the primary key rather than switching to auto-incrementing integers, so every existing `rackDetailUrl(id)` / `itemDetailUrl(id)` link, QR code payload, and CSV export keeps working untouched. `createdAt` / `timestamp` are `BigIntegerField` storing the exact same epoch-millisecond number `Date.now()` produces client-side — not Django's `DateTimeField`, which would change the wire format. `images` stays a plain `JSONField(list)` of base64 strings, identical to how the frontend already stores it — no file-backed model, no upload endpoint required to get started. The tiny `epoch_ms()` helper is duplicated in both apps' models.py rather than shared from a third location — it's three lines, and importing across apps just for that would add coupling for no real benefit.

```python
# accounts/models.py
import time
from django.db import models


def epoch_ms():
    """Matches the frontend's Date.now() exactly — epoch milliseconds."""
    return int(time.time() * 1000)


PERMISSION_MODULES = ["storage", "items", "rooms", "users", "roles"]

class Role(models.Model):
    id = models.CharField(max_length=20, primary_key=True)        # e.g. "RL-1002"
    name = models.CharField(max_length=255)
    description = models.TextField(blank=True, default="")
    # Exact shape the frontend already edits: {"storage": {"create":T,"read":T,"update":T,"delete":T}, ...}
    permissions = models.JSONField(default=dict)
    created_at = models.BigIntegerField(default=epoch_ms)

    class Meta:
        db_table = "roles"


class User(models.Model):
    id = models.CharField(max_length=20, primary_key=True)        # e.g. "US-1002"
    name = models.CharField(max_length=255)
    username = models.CharField(max_length=150, unique=True)
    email = models.EmailField()                                   # required — see Section 2.4
    phone = models.CharField(max_length=30, blank=True, default="")
    role = models.ForeignKey(Role, on_delete=models.SET_NULL, null=True, blank=True, related_name="users")
    status = models.CharField(
        max_length=20,
        choices=[("active", "Active"), ("inactive", "Inactive")],
        default="active",
    )
    created_at = models.BigIntegerField(default=epoch_ms)

    class Meta:
        db_table = "users"
```

```python
# storage_rack/models.py
import time
from django.db import models


def epoch_ms():
    """Matches the frontend's Date.now() exactly — epoch milliseconds."""
    return int(time.time() * 1000)


class Location(models.Model):
    id = models.CharField(max_length=20, primary_key=True)        # e.g. "SR-1002"
    name = models.CharField(max_length=255)
    location = models.CharField(max_length=255, blank=True, default="")
    created_at = models.BigIntegerField(default=epoch_ms)

    class Meta:
        db_table = "locations"

    def __str__(self):
        return self.name


STORAGE_TYPES = [
    ("rack", "Rack"), ("cupboard", "Cupboard"), ("bureau", "Bureau"),
    ("wardrobe", "Wardrobe"), ("shelf", "Shelf"), ("cabinet", "Cabinet"),
    ("box", "Box"), ("custom", "Custom"),
]

class StorageUnit(models.Model):
    id = models.CharField(max_length=20, primary_key=True)        # e.g. "ST-1008"
    name = models.CharField(max_length=255)
    store_room = models.ForeignKey(Location, on_delete=models.PROTECT, related_name="storage_units")
    type = models.CharField(max_length=20, choices=STORAGE_TYPES)
    tags = models.JSONField(default=list)                         # list[str], exact match
    images = models.JSONField(default=list)                       # list[str] of base64 data-URIs, exact match
    created_at = models.BigIntegerField(default=epoch_ms)

    class Meta:
        db_table = "storage_units"

    def __str__(self):
        return f"{self.id} {self.name}"


ITEM_STATUS = [("in_store", "In Store"), ("in_use", "In Use"), ("removed", "Removed")]

class Item(models.Model):
    id = models.CharField(max_length=20, primary_key=True)        # e.g. "IT-1013"
    name = models.CharField(max_length=255)
    tags = models.JSONField(default=list)
    quantity = models.PositiveIntegerField(default=1)
    remarks = models.TextField(blank=True, default="")
    storage = models.ForeignKey(
        StorageUnit, on_delete=models.SET_NULL, null=True, blank=True, related_name="items"
    )
    images = models.JSONField(default=list)                       # list[str] of base64 data-URIs, exact match
    status = models.CharField(max_length=20, choices=ITEM_STATUS, default="in_store")
    created_at = models.BigIntegerField(default=epoch_ms)

    class Meta:
        db_table = "items"

    def __str__(self):
        return f"{self.id} {self.name}"


LOG_TYPES = [
    ("in", "In"), ("out", "Out"),
    ("in_store", "In Store"), ("in_use", "In Use"), ("removed", "Removed"),
]
```

```
class ItemLogEntry(models.Model):
    id = models.CharField(max_length=30, primary_key=True)
    timestamp = models.BigIntegerField(default=epoch_ms)          # epoch ms, exact match
    type = models.CharField(max_length=20, choices=LOG_TYPES)
    item_name = models.CharField(max_length=255)                  # itemName — denormalized snapshot, exact match
    quantity = models.PositiveIntegerField()
    rack = models.ForeignKey(
        StorageUnit, on_delete=models.SET_NULL, null=True, blank=True, related_name="log_entries"
    )
    rack_name = models.CharField(max_length=255, blank=True, default="")   # rackName — denormalized snapshot
    path = models.CharField(max_length=500, null=True, blank=True)         # legacy field, always null today — kept for exact
parity
    user = models.CharField(max_length=150)                       # exact match: raw username string, not a FK to accounts.User

    class Meta:
        db_table = "item_log"
        ordering = ["-timestamp"]
        verbose_name_plural = "item log entries"
```

> **Where this model set is *not* a 1:1 JSON mirror, and why:** `store_room` / `storage` / `rack` / `role` are Django `ForeignKey`s rather than plain ID strings — that's the direct relational-database equivalent of the frontend's `storeRoomId` / `storageId` / `rackId` / `roleId` string fields (the serializers in Section 6.2 expose them back out as those exact same ID strings, so nothing changes from the frontend's point of view). `Role` and `User` deliberately live together in `accounts` so that one FK stays intra-app — `storage_rack`'s four models never import from `accounts`, and `ItemLogEntry.user` stays a plain username string (matching the frontend) rather than a FK to `accounts.User`, so the model layer has zero coupling between the two apps in either direction. The frontend's Users table (Section 2.4) now carries `phone` and `password` fields too; `phone` is reflected above, but `password` is only a **plaintext demo value** checked by `js/login.js` directly against `localStorage` — the Users page's invite/reset action only sends a request; it never writes this field itself, so there is currently no in-UI path that actually populates it. The field was never hashed, salted, or sent anywhere either way, so it is not fit to migrate as-is. A real login still needs its own, properly hashed password field (e.g. Django's own `AbstractUser`, or a bolted-on `password_hash = models.CharField(max_length=255)`) populated through a real set-password/invite endpoint — the frontend's plaintext field only establishes that the shape (one password per user, set after the fact, gating login) is already proven out in the UI; the value itself must not be carried over.

6.2 DRF Serializers

One serializer per model, with explicit field renames (`source=`) so the JSON over the wire uses the exact same camelCase field names the frontend already expects (`storeRoomId`, `storageId`, etc.) even though the Python/DB side stays snake_case. Split the same way as the models: `accounts/serializers.py`, `storage_rack/serializers.py`.

```
# accounts/serializers.py
from rest_framework import serializers
from .models import Role, User


class RoleSerializer(serializers.ModelSerializer):
    class Meta:
        model = Role
        fields = ["id", "name", "description", "permissions", "created_at"]


class UserSerializer(serializers.ModelSerializer):
    roleId = serializers.CharField(source="role_id", required=False, allow_null=True)

    class Meta:
        model = User
        fields = ["id", "name", "username", "email", "phone", "roleId", "status", "created_at"]
```

```python
# storage_rack/serializers.py
from rest_framework import serializers
from .models import Location, StorageUnit, Item, ItemLogEntry


class LocationSerializer(serializers.ModelSerializer):
    class Meta:
        model = Location
        fields = ["id", "name", "location", "created_at"]


class StorageUnitSerializer(serializers.ModelSerializer):
    storeRoomId = serializers.CharField(source="store_room_id")
    # images is already a plain JSONField(list) on the model — ModelSerializer
    # passes it through as-is, exactly like tags, with no nested serializer needed.

    class Meta:
        model = StorageUnit
        fields = ["id", "name", "storeRoomId", "type", "tags", "images", "created_at"]


class ItemSerializer(serializers.ModelSerializer):
    storageId = serializers.CharField(source="storage_id", required=False, allow_null=True)

    class Meta:
        model = Item
        fields = ["id", "name", "tags", "quantity", "remarks", "storageId", "images", "status", "created_at"]


class ItemLogEntrySerializer(serializers.ModelSerializer):
    itemName = serializers.CharField(source="item_name")
    rackId = serializers.CharField(source="rack_id", allow_null=True)
    rackName = serializers.CharField(source="rack_name")

    class Meta:
        model = ItemLogEntry
        fields = ["id", "timestamp", "type", "itemName", "quantity", "rackId", "rackName", "path", "user"]
```

6.3 Permission Enforcement

The Roles & Permissions matrix (Section 2.5) is fully modeled and editable in the UI today, but nothing currently reads or enforces it — this class is what finally makes it do something, by checking the requesting user's role against the exact `{module: {create,read,update,delete}}` JSON shape already in `Role.permissions`. It lives in `accounts`, since it's fundamentally about `Role` — `storage_rack`'s views import it from there rather than each app keeping its own copy.

```python
# accounts/permissions.py
from rest_framework.permissions import BasePermission

ACTION_TO_PERM = {
    "list": "read", "retrieve": "read",
    "create": "create", "update": "update", "partial_update": "update",
    "destroy": "delete",
}

class HasModulePermission(BasePermission):
    """Checks request.user.role.permissions[view.module][action] — the exact
    JSON structure the Roles & Permissions page already edits (Section 2.5).
    Both apps' views use this; storage_rack has no Role model of its own."""

    def has_permission(self, request, view):
        if not request.user or not request.user.is_authenticated:
            return False
        module = getattr(view, "module", None)
        if not module:
            return True
        role = request.user.role
        if not role:
            return False
        perm_key = ACTION_TO_PERM.get(view.action, "read")
        return bool(role.permissions.get(module, {}).get(perm_key))
```

6.4 Services Layer

Multi-step mutations and cross-cutting checks — the kind of logic that today lives inline inside a view method — move into a `services.py` per app instead, so the same operation can be called identically from a view, the Django admin, or a future management command, and is independently testable without spinning up a request. Each app's services only ever touch that app's own models — `accounts/services.py` never imports from `storage_rack`, and vice versa, mirroring the no-coupling rule from Section 6.1.

```python
# accounts/services.py
from django.utils.crypto import get_random_string
from .models import User


def send_password_reset_invite(user):
    """The "Send Invite" / table reset-password action (Section 5's Invite /
    Reset Password flow). Generates a single-use token and emails a link
    containing it — the token itself is never returned to the caller,
    mirroring the Forgot Password flow's "never return the code to the
    client" rule."""
    if not user.email:
        raise ValueError("User has no email on file.")
    token = get_random_string(40)
    user.reset_token = token          # see the note below this block
    user.save(update_fields=["reset_token"])
    # send_mail(...) — real email delivery goes here; the frontend never
    # implements anything past the confirmation + toast (Section 5).
    return token


def set_password_from_token(token, new_password):
    """What the recipient's link actually does when they follow it — the
    half of the invite flow the current frontend has no page for at all."""
    user = User.objects.get(reset_token=token)
    user.set_password(new_password)   # requires AbstractUser — see Section 6.6
    user.reset_token = ""
    user.save(update_fields=["password", "reset_token"])
    return user


def role_in_use_count(role):
    """Backs the Role delete safeguard (Section 5, Section 7) and the
    "userCount" figure the Roles & Permissions table already shows."""
    return role.users.count()
```

> `send_password_reset_invite` / `set_password_from_token` above reference a `reset_token` field that isn't in the `User` model shown in Section 6.1 — add it there (e.g. `reset_token = models.CharField(max_length=64, blank=True, default="")`) if you implement the invite flow this way. That's one reasonable approach for a quick build, not the only one — a separate `PasswordResetToken` model with its own expiry timestamp is the more conventional choice once this needs to be production-hardened rather than a demo-matching first pass.

```python
# storage_rack/services.py
from django.utils import timezone
from .models import Item, ItemLogEntry


def archive_item(item, username):
    """ItemViewSet.archive (Section 6.5) — marks the item removed and
    records the change, matching archiveGlobalItem (Section 4)."""
    item.status = "removed"
    item.save()
    ItemLogEntry.objects.create(
        id=f"LG-{int(timezone.now().timestamp())}", type="removed",
        item_name=item.name, quantity=item.quantity,
        rack=item.storage, rack_name=item.storage.name if item.storage else "",
        user=username,
    )
    return item


def restore_item(item):
    """ItemViewSet.restore (Section 6.5) — matches restoreGlobalItem (Section 4)."""
    item.status = "in_store"
    item.save()
    return item


def move_item(item, quantity, storage_id, status):
    """ItemViewSet.move (Section 6.5) — the quantity-split behaviour from
    the Move / Use Item flow (Section 5), unchanged by the app split."""
    if quantity >= item.quantity:
        item.storage_id = storage_id
        item.status = status
        item.save()
        return item, None
    item.quantity -= quantity
    item.save()
    split = Item.objects.create(
        id=f"IT-{int(timezone.now().timestamp())}", name=item.name,
        tags=item.tags, quantity=quantity, remarks=item.remarks,
        storage_id=storage_id, status=status,
    )
    return item, split


def location_in_use_count(location):
    """Backs the Location delete safeguard (Section 5, Section 7)."""
    return location.storage_units.count()
```

6.5 DRF Views

One `ModelViewSet` per resource, now thin — each view's body is the HTTP concern (query params, status codes, serialization) while the actual logic lives in the services layer above. `StorageUnitViewSet` and `ItemViewSet` add the same search/filter query params the frontend's `applyRacksFilters` / `applyItemFilters` already compute client-side (Section 4), just moved server-side.

```python
# accounts/views.py
from rest_framework import viewsets, status
from rest_framework.decorators import action
from rest_framework.response import Response
from .models import Role, User
from .serializers import RoleSerializer, UserSerializer
from .permissions import HasModulePermission
from . import services


class RoleViewSet(viewsets.ModelViewSet):
    queryset = Role.objects.all()
    serializer_class = RoleSerializer
    permission_classes = [HasModulePermission]
    module = "roles"

    def destroy(self, request, *args, **kwargs):
        role = self.get_object()
        if services.role_in_use_count(role):
            return Response(
                {"detail": "Reassign its user(s) to another role first."},
                status=status.HTTP_409_CONFLICT,
            )
        return super().destroy(request, *args, **kwargs)


class UserViewSet(viewsets.ModelViewSet):
    queryset = User.objects.all()
    serializer_class = UserSerializer
    permission_classes = [HasModulePermission]
    module = "users"

    @action(detail=True, methods=["post"])
    def invite(self, request, pk=None):
        """The "Send Invite" / table reset-password action — see the Invite
        / Reset Password flow in Section 5. 400 if the user has no email."""
        user = self.get_object()
        try:
            services.send_password_reset_invite(user)
        except ValueError as exc:
            return Response({"detail": str(exc)}, status=400)
        return Response({"detail": f"Invitation sent to {user.email}"})
```

```python
# storage_rack/views.py
from rest_framework import viewsets, status
from rest_framework.decorators import action
from rest_framework.response import Response
from .models import Location, StorageUnit, Item, ItemLogEntry
from .serializers import (
    LocationSerializer, StorageUnitSerializer, ItemSerializer, ItemLogEntrySerializer,
)
from accounts.permissions import HasModulePermission
from . import services


class LocationViewSet(viewsets.ModelViewSet):
    queryset = Location.objects.all()
    serializer_class = LocationSerializer
    permission_classes = [HasModulePermission]
    module = "rooms"

    def destroy(self, request, *args, **kwargs):
        location = self.get_object()
        if services.location_in_use_count(location):
            return Response(
                {"detail": "Move or delete its storage units first."},
                status=status.HTTP_409_CONFLICT,
            )
        return super().destroy(request, *args, **kwargs)


class StorageUnitViewSet(viewsets.ModelViewSet):
    queryset = StorageUnit.objects.all()
    serializer_class = StorageUnitSerializer
    permission_classes = [HasModulePermission]
    module = "storage"

    def get_queryset(self):
        qs = super().get_queryset()
        p = self.request.query_params
        if p.get("search"):
            qs = qs.filter(name__icontains=p["search"])
        if p.get("room"):
            qs = qs.filter(store_room_id=p["room"])
        if p.get("type"):
            qs = qs.filter(type=p["type"])
        return qs


class ItemViewSet(viewsets.ModelViewSet):
    queryset = Item.objects.all()
    serializer_class = ItemSerializer
    permission_classes = [HasModulePermission]
    module = "items"

    def get_queryset(self):
        qs = super().get_queryset()
        p = self.request.query_params
        if p.get("status"):
            qs = qs.filter(status=p["status"])
        if p.get("storageId"):
            qs = qs.filter(storage_id=p["storageId"])
        return qs

    @action(detail=True, methods=["post"])
    def archive(self, request, pk=None):
        item = services.archive_item(self.get_object(), request.user.username)
        return Response(self.get_serializer(item).data)

    @action(detail=True, methods=["post"])
    def restore(self, request, pk=None):
        item = services.restore_item(self.get_object())
        return Response(self.get_serializer(item).data)
```

```
    @action(detail=True, methods=["post"])
    def move(self, request, pk=None):
        """Splits off `quantity` units into `storageId` with `status`,
        mirroring openMoveItemModal's split/move behaviour (Section 4) —
        the branching itself now lives in services.move_item."""
        item, _split = services.move_item(
            self.get_object(),
            quantity=int(request.data["quantity"]),
            storage_id=request.data.get("storageId"),
            status=request.data.get("status", self.get_object().status),
        )
        return Response(self.get_serializer(item).data)


class ItemLogViewSet(viewsets.ReadOnlyModelViewSet):
    queryset = ItemLogEntry.objects.all()
    serializer_class = ItemLogEntrySerializer
    permission_classes = [HasModulePermission]
    module = "items"

    def get_queryset(self):
        qs = super().get_queryset()
        p = self.request.query_params
        if p.get("type"):
            qs = qs.filter(type=p["type"])
        if p.get("itemName"):
            qs = qs.filter(item_name__icontains=p["itemName"])
        if p.get("storageId"):
            qs = qs.filter(rack_id=p["storageId"])
        if p.get("user"):
            qs = qs.filter(user_id=p["user"])
        return qs
```

> `from accounts.permissions import HasModulePermission` at the top of `storage_rack/views.py` is the one deliberate cross-app import in the whole backend — everything else about the split in Section 6.1's note holds; this single line is the cost of not duplicating the permission class in two places.

6.6 Authentication & URL Routing

Replaces the single hardcoded `admin` / `admin123` credential in `js/login.js` with a real token per `User` row via DRF's built-in `authtoken` app. Login/logout live in `accounts`, same as the models they operate on.

> `django.contrib.auth.authenticate()`, `TokenAuthentication`, and `request.user` all require the user model to be wired into Django's auth system — the exact `User` model in Section 6.1 is data-storage only and isn't yet. The smallest change that makes this code actually run: make `User` extend `django.contrib.auth.models.AbstractUser` instead of `models.Model`, add `AUTH_USER_MODEL = "accounts.User"` to `settings.py`, and drop the now-redundant `username` / `email` field declarations (`AbstractUser` already provides them, plus `password`, `is_staff`, `is_active`, etc. — fields beyond Section 2.4's exact shape, but required for login to work at all).

```
# accounts/auth_views.py
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework.authtoken.models import Token
from django.contrib.auth import authenticate

class LoginView(APIView):
    permission_classes = []

    def post(self, request):
        user = authenticate(
            username=request.data.get("username"),
            password=request.data.get("password"),
        )
        if not user:
            return Response({"detail": "Invalid credentials."}, status=401)
        token, _ = Token.objects.get_or_create(user=user)
        return Response({
            "token": token.key,
            "user": {"id": user.id, "name": user.name, "roleId": user.role_id},
        })


class LogoutView(APIView):
    def post(self, request):
        request.user.auth_token.delete()
        return Response(status=204)
```

```
# accounts/urls.py
from django.urls import path, include
from rest_framework.routers import DefaultRouter
from .views import RoleViewSet, UserViewSet
from .auth_views import LoginView, LogoutView

router = DefaultRouter()
router.register("roles", RoleViewSet)
router.register("users", UserViewSet)

urlpatterns = [
    path("", include(router.urls)),
    path("auth/login/", LoginView.as_view()),
    path("auth/logout/", LogoutView.as_view()),
]
```

```
# storage_rack/urls.py
from django.urls import path, include
from rest_framework.routers import DefaultRouter
from .views import LocationViewSet, StorageUnitViewSet, ItemViewSet, ItemLogViewSet

router = DefaultRouter()
router.register("locations", LocationViewSet)
router.register("storages", StorageUnitViewSet)
router.register("items", ItemViewSet)
router.register("log", ItemLogViewSet)

urlpatterns = [
    path("", include(router.urls)),
]
```

```
# backend/urls.py — project root
from django.urls import path, include

urlpatterns = [
    path("api/", include("accounts.urls")),
    path("api/", include("storage_rack.urls")),
]
```

Mounting both apps' `urls.py` under the same `"api/"` prefix from the project root reproduces exactly the same endpoint paths the single-app version had — `/api/users/`, `/api/roles/`, `/api/auth/login/` from `accounts`; `/api/storages/`, `/api/items/`, `/api/locations/`, `/api/log/` from `storage_rack` — so nothing about the frontend's call sites (Section 3, Section 4's Pages → API Call Map) needs to change just because the backend is now two apps instead of one.

RESOURCE	ENDPOINTS	REPLACES (CLIENT-SIDE)
Storage Units	`GET/POST /api/storages/` `GET/PUT/PATCH/DELETE /api/storages/{id}/`	`loadRacks` / `saveRacks`; `PATCH` backs the per-field click-to-edit on Storage Detail
Items	`GET/POST /api/items/` `GET/PUT/PATCH/DELETE /api/items/{id}/` `POST /api/items/{id}/archive/` · `/restore/` · `/move/`	`loadGlobalItems` / `saveGlobalItems`, `archiveGlobalItem`, `restoreGlobalItem`, `openMoveItemModal`'s split/move flow
Locations	`GET/POST /api/locations/` `PUT/DELETE /api/locations/{id}/`	`loadRooms` / `saveRooms` (delete returns 409 if any storage still references it, same guard `storerooms.js` already does client-side)
Users	`GET/POST /api/users/` `PUT/DELETE /api/users/{id}/` `POST /api/users/{id}/invite/` `POST /api/auth/login/` · `/logout/`	`loadUsers` / `saveUsers`; replaces the hardcoded demo login entirely; `invite/` backs "Send Invite" / the table's reset-password action (Section 5)
Roles	`GET/POST /api/roles/` `PUT/DELETE /api/roles/{id}/`	`loadRoles` / `saveRoles`
IN/OUT Log	`GET /api/log/` (read-only; filters: `type`, `itemName`, `storageId`, `user`)	`loadLog`; entries are written server-side as a side effect of item mutations, never POSTed directly by the client

6.7 Project Setup

```
# settings.py — essentials only
INSTALLED_APPS = [
    ...
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "rest_framework",
    "rest_framework.authtoken",
    "corsheaders",
    "accounts",                          # Users & Roles app
    "storage_rack",                      # Storages, Items, Locations, IN/OUT Log app
]

AUTH_USER_MODEL = "accounts.User"

MIDDLEWARE = [
    "corsheaders.middleware.CorsMiddleware",
    ...
]

REST_FRAMEWORK = {
    "DEFAULT_AUTHENTICATION_CLASSES": [
        "rest_framework.authentication.TokenAuthentication",
    ],
    "DEFAULT_PERMISSION_CLASSES": [
        "rest_framework.permissions.IsAuthenticated",
    ],
}

CORS_ALLOWED_ORIGINS = ["http://localhost:8765"]  # wherever the existing frontend is served from
```

```
# requirements.txt
Django>=5.0
djangorestframework>=3.15
django-cors-headers>=4.3
```

No `Pillow`, `MEDIA_URL`, or `MEDIA_ROOT` needed to get started — `images` is a plain `JSONField` on `StorageUnit` / `Item` (Section 6.1), so base64 data-URIs round-trip exactly as the frontend already sends them, no file storage required. Add those later only if you choose to normalize into real file uploads (optional — see Section 6.8).

Standard bring-up from there: `django-admin startproject backend`, then `python manage.py startapp accounts` and `python manage.py startapp storage_rack` (two apps, not one), drop the files above into their matching app, then `python manage.py makemigrations accounts storage_rack && python manage.py migrate && python manage.py createsuperuser && python manage.py runserver`.

Folder structure for the two-app split — this redesigns the single-app backend that was previously scaffolded and verified end to end (`pip install -r requirements.txt`, `manage.py check`, `makemigrations`, `migrate` against a real SQLite database, dev server booted cleanly). That verification covered the original single-app layout; splitting an already-working app into two is a mechanical, low-risk refactor (move files, split `INSTALLED_APPS`, update the handful of imports shown above) rather than new behavior, but the two-app layout below has not itself been re-scaffolded and re-run on disk — treat it as a documented, internally-consistent proposal rather than a freshly re-confirmed one.

```
backend/
├── manage.py
├── requirements.txt              Django, djangorestframework, django-cors-headers (§6.7)
├── .gitignore                    excludes __pycache__/, db.sqlite3, .env, venv/
├── db.sqlite3                    created by "migrate" — gitignored, not committed
├── backend/                      — project config (from "django-admin startproject backend")
│   ├── __init__.py
│   ├── settings.py                INSTALLED_APPS, MIDDLEWARE, REST_FRAMEWORK, AUTH_USER_MODEL, CORS_ALLOWED_ORIGINS (§6.6–6.7)
│   ├── urls.py                    mounts accounts.urls and storage_rack.urls, both under "api/" (§6.6)
│   ├── wsgi.py
│   └── asgi.py
├── accounts/                     — Users & Roles app (from "python manage.py startapp accounts")
│   ├── __init__.py
│   ├── models.py                  Role, User (extends AbstractUser) (§6.1)
│   ├── serializers.py             RoleSerializer, UserSerializer, camelCase field mapping (§6.2)
│   ├── permissions.py             HasModulePermission — checks role.permissions[module][action]; imported by storage_rack too
(§6.3)
│   ├── services.py                invite/reset-password logic, role in-use count (§6.4)
│   ├── views.py                   RoleViewSet, UserViewSet + invite action (§6.5)
│   ├── auth_views.py              LoginView, LogoutView — replaces the hardcoded demo credential (§6.6)
│   ├── urls.py                    DefaultRouter for roles/users + auth routes (§6.6)
│   ├── admin.py                   registers Role and User with the Django admin site
│   ├── apps.py
│   └── migrations/
│       ├── __init__.py
│       └── 0001_initial.py         generated by "python manage.py makemigrations accounts"
└── storage_rack/                 — Storages, Items, Locations & IN/OUT Log app (from "python manage.py startapp storage_rack")
    ├── __init__.py
    ├── models.py                  Location, StorageUnit, Item, ItemLogEntry (§6.1)
    ├── serializers.py             one DRF serializer per model, camelCase field mapping (§6.2)
    ├── services.py                archive/restore/move logic, location in-use count (§6.4)
    ├── views.py                   one ModelViewSet per resource, + archive/restore/move actions (§6.5)
    ├── urls.py                    DefaultRouter for locations/storages/items/log (§6.6)
    ├── admin.py                   registers the remaining four models with the Django admin site
    ├── apps.py
    └── migrations/
        ├── __init__.py
        └── 0001_initial.py         generated by "python manage.py makemigrations storage_rack"
```

6.8 Migration Path

Keep the frontend changes small by preserving function names and JSON shapes end to end:

- Request/response bodies match the field names in Section 2 exactly — no reshaping needed in the page scripts that already consume these objects.
- Each `load*()` in `common.js` becomes a `fetch()` to its `GET` list endpoint; each `save*(data)` — which today rewrites the entire collection to `localStorage` on every single mutation — is replaced by the one targeted `POST` / `PUT` / `PATCH` / `DELETE` call that the mutation actually needs.
- `window.onRackDataChanged` / `window.onItemDataChanged` (Section 5) keep their exact role — just fired from a fetch `.then()` instead of right after a synchronous `localStorage.setItem`.
- `images` needs no migration at all day one — it's a `JSONField(list)` on the model (Section 6.1), so the frontend's existing base64 array round-trips through the API unchanged. Normalizing to real file uploads (a separate `StorageImage` / `ItemImage` model, `multipart/form-data`, returned URLs instead of inline data-URIs) is a worthwhile optimization once image volume grows, but it's an opt-in improvement, not a requirement to launch.
- The two-app split (Section 6.1) is purely a backend organization choice — it changes no request/response shape and no endpoint path (Section 6.6), so it has zero effect on this migration path or on any frontend code.

Generated from the current working tree of D:\storage_rack.